

AI Trip Planner

Multi-Agent System built with LangGraph, Flask, and Live APIs
OpenAI GPT-4o-mini | OpenRouter Llama 3.3-70B | Serper | OpenWeatherMap

SECTION 1

What Does This App Do?

A user types where they want to go and their budget. The system automatically runs multiple AI agents in the background — fetching live weather, finding hotels, searching transport, discovering attractions — and produces a complete day-by-day trip plan with a downloadable PDF report. No manual research needed.

Step 1 - User fills the form or chats with Trippy (AI chatbot)

Enters source, destination, dates, budget, travelers, hotel & transport preferences.

Step 2 - System validates and collects live data

4 agents fetch real weather, transport, hotel listings, and tourist attractions from live APIs.

Step 3 - AI writes a complete day-by-day itinerary

GPT-4o-mini uses all real data to generate a weather-aware, budget-aware trip plan.

Step 4 - Plan is reviewed, scored, and saved as PDF

Another AI reviews quality. A third AI scores 0-100. PDF auto-generated and downloadable.

SECTION 2

Folder Structure

Every file has one clear job. The project is organized so each layer is separate.

File	What it does
app.py	Flask web server — defines all HTTP endpoints, runs the workflow
workflow.py	Defines the LangGraph graph — connects all 11 agents in order
state.py	TripState TypedDict — shared data container all agents read/write
guardrails.py	Input validation — runs BEFORE the workflow starts
config.py	API keys and settings (loaded from .env)
requirements.txt	Python packages needed to run the app
.env	Your private API keys — never share this file
start.py	Launch script — sets UTF-8 encoding then starts Flask
architecture.html/pdf	Visual architecture flow diagram
PRESENTATION.html/pdf	This document
agents/ (11 files)	One file per agent — each agent does ONE job

templates/index.html	The web UI — form, results dashboard, chatbot
memory/	Stores user_memory.json — past trips and preferences
output/	Generated PDF files saved here

SECTION 3

How to Run the App

Follow these steps in order. Steps 1-4 are one-time setup.

Step 1 - Install Python 3.11+

Download from python.org. Verify: `python --version`

Step 2 - Install all dependencies

In the `trip_planner/` folder run: `pip install -r requirements.txt`

Step 3 - Configure API keys

Fill in `.env` file: `OPENAI_API_KEY`, `SERPER_API_KEY`, `OPENWEATHER_API_KEY`, `OPENROUTER_API_KEY`

Step 4 - Start the server

Run: `python -X utf8 start.py` Then open `http://localhost:8000`

Step 5 - Plan a trip

Fill the form or click the chat bubble to talk to Trippy. Click Plan My Trip. Wait 30-60 sec.

Step 6 - Download PDF

Click Download PDF button after the plan is generated.

SECTION 4

All 11 Agents Explained

Each agent is in its own file under `agents/`. Each does one job and writes its result to `TripState`.

Orchestrator

`agents/orchestrator.py` | Rule-Based Logic

The brain of the system. NOT an AI — pure Python if/else. Runs at 3 decision checkpoints: validate input, check data conflicts, approve final plan. Routes the workflow forward or stops it.

Memory Agent

`agents/memory_agent.py` | Local JSON File

Reads and writes `user_memory.json`. On start: loads past preferences. After plan approved: saves this trip so future trips remember the user preferences.

Weather Agent

`agents/weather_agent.py` | OpenWeatherMap API

Gets real daily forecast, min/max temperature, and rainfall risk level for the destination. Triggers weather warning popup if heavy rain or temperature $\geq 38^{\circ}\text{C}$.

Transport Agent

`agents/transport_agent.py` | Serper (Google) API

Searches real flights, trains, or buses. Finds the nearest departure hub city using geocoding. Sets `no_tickets_available` flag if no options found.

Hotel Agent

`agents/hotel_agent.py` | Serper (Google) API

Finds real hotels matching user preference (beach resort, heritage, business etc.) and budget tier. Calculates total cost based on rooms and nights.

Places Agent

`agents/places_agent.py` | Serper (Google) API

Fetches real tourist spots, restaurants, and nightlife. Also finds nearby day-trip destinations within 3 hours. Switches to indoor-only results if heavy rain expected.

Budget Agent

`agents/budget_agent.py` | Rule-Based Math (No LLM)

Adds up all costs. If over budget: tries cheaper hotel tier first, then cheaper transport. Sets `budget_exceeded` flag and shows popup in UI if still over limit.

Itinerary Agent

`agents/itinerary_agent.py` | OpenAI GPT-4o-mini

Uses GPT-4o-mini to write the full day-by-day plan using REAL places. Includes must-visit places and schedules nearby day trips every 2 days. Validates output for hallucinated place names.

Review Agent

`agents/review_agent.py` | OpenRouter Llama 3.3-70B

Sends the full trip plan to Llama 3.3 as a travel consultant. Returns a review paragraph, list of issues, list of highlights, and verdict: `approved` or `needs_retry`.

Evaluation Agent

`agents/evaluation_agent.py` | OpenRouter Llama 3.3-70B

Scores the plan out of 100 across 5 areas: Budget Fit (25), Completeness (25), Weather Alignment (20), Data Quality (15), Transport Logic (15). Returns grade `A/B/C/D`.

PDF Generator

`agents/pdf_agent.py` | ReportLab (Local)

Generates a multi-page PDF: cover page, budget table, day-wise itinerary, hotel and transport details, weather summary, packing checklist, emergency contacts.

SECTION 5

Key Design Decisions

Q: Why LangGraph?

A: Gives a visual, inspectable graph. Each agent only reads/writes `TripState` — they do not call each other. This makes each agent independently testable and replaceable.

Q: Why 3 checkpoints instead of 1?

A: Gate 1 stops bad input before any API calls (saves cost). Gate 2 catches conflicts before the expensive LLM call. Gate 3 verifies LLM output before writing to disk.

Q: Why rule-based Budget Agent?

A: Budget math must be exact and deterministic. Simple math is always correct, always fast, and free. LLMs are only used where natural language generation adds real value.

Q: Why two different LLMs?

A: GPT-4o-mini is best for structured generation (itinerary). Llama 3.3 via OpenRouter is free-tier accessible and good for review and evaluation. If one fails, the other is fallback.

Q: Why TripState?

A: Instead of passing 30 arguments between agents, one shared dictionary holds everything. Every agent reads what it needs and writes its result back. Standard multi-agent pattern.

SECTION 6

Technology Stack

Layer	Technology	Purpose
Web Server	Flask (Python)	HTTP requests, serves UI, routes to workflow
Workflow Engine	LangGraph	Agent graph, state management, conditional routing
LLM Interface	LangChain	Standardized way to call OpenAI and OpenRouter
Primary LLM	OpenAI GPT-4o-mini	Itinerary generation
Fallback LLM	OpenRouter Llama 3.3	Review, evaluation, chatbot (free tier available)
Places/Hotels/Transport	Serper API	Google Search API — real results for any city
Weather	OpenWeatherMap	Real-time forecast, temperature, rainfall
PDF Generation	ReportLab	Creates PDFs locally, no external service needed
Frontend	HTML / CSS / JS	Web UI with date picker, no heavy framework
Memory	JSON file	Saves user history between sessions